Data Normalization & LLM Extraction Assignment

1. Objective and Context

As a Python developer intern, a core part of your role will involve taking messy, unstructured, and highly complex e-commerce data and transforming it into clean, structured datasets suitable for analytics and business intelligence using agents and other machine learning techniques.

Retailers and telecom providers frequently bundle products with highly conditional promotional text (e.g., "Get $800 off with a trade-in," "Free smartwatch with a new line"). Your task is to process a raw JSON payload scraped from a retailer's website and output a strictly formatted CSV file that accurately captures these intricate promotional details.

2. The Input Data

You will be provided with a raw JSON file (e.g., input_data.json) containing an array of product objects. Each object represents a specific device and variant combination.

Within each product object, you will find:

- **Basic Metadata:** Product name, brand, variant (storage size), stock status, and shipping info.
- **Pricing:** Base retail and monthly installment pricing.
- **Offers Array:** Unstructured text snippets detailing various promotions (switcher incentives, new line bonuses, credit card or finance offers, and more).
- **Accessory/Add-on Arrays:** Lists of supplementary items (watches, earbuds, cases, chargers) that may include their own bundled discounts.

3. Output Requirements

Your script must output a single CSV file. Your normalization logic must evaluate the raw data and accurately extract information into the following exact columns. **Every column must be present in your final CSV**, even if the value is null or empty for a specific row.

Required CSV Columns:

1. PLAN TYPE
2. PRODUCT NAME
3. BRAND
4. VARIANT
5. BASE
6. MSRP
7. RESELLER LIST PRICE
8. SWITCHER
9. SWITCHER_OFFER_TEXT
10. UNIQUE DISCOUNT
11. FREE MEMORY UPGRADE
12. BOGO OFFER
13. Max TI *(Maximum Trade-In Value)*
14. Trade-In Value Type (Enhanced/ Base)
15. LINE TYPE *(e.g., New Line vs. Upgrade)*
16. RATEPLAN DURATION (MONTHS)
17. SHIPMENT
18. STOCK
19. PWP/GWP OFFERS *(Pay-With-Purchase / Gift-With-Purchase)*
20. Cleaned PWP/GWP OFFERS
21. PWP/GWP OFFER $ VALUE
22. SUBSCRIPTION BUNDLE OFFERS
23. SUBSCRIPTION BUNDLE OFFER $ VALUE
24. SERVICE LINE OFFERS
25. SERVICE LINE OFFER $ VALUE
26. ADDITIONAL OFFERS
27. ADDITIONAL OFFER $ VALUE
28. FINANCE OFFER DESCRIPTION 1
29. FINANCE OFFER $ VALUE

Detailed Example: Interpreting the Apple iPhone 17 Pro Max (256 GB) Record

The following breakdown demonstrates how a single JSON record from Example.json must be analyzed and mapped to the required output schema. A core requirement of this normalization task is evaluating the exact same product twice: once assuming the customer is adding a **New Line**, and once assuming they are processing an **Upgrade** (existing customer).

1. Base Product Fields

- **Interpretation:** Extract the fundamental device characteristics directly from the top-level JSON keys without applying any promotional logic.
- **Expected Output (Applies to both Line Types):**
    - PRODUCT NAME: Apple iPhone 17 Pro Max
    - BRAND: Apple
    - VARIANT: 256 GB
    - MSRP: 1199.99 (Derived directly from pricing.retail)

2. Unique Discount & Service Line Offers (The Impact of Line Types)

- **Source Text:** The JSON contains an overarching offer block stating: *"Eligibility: New customers activating new line... iPhone 17 Pro Max 256GB for $9.43/mo. for 36 mos. after up to $660.36 in bill credits... plus an additional $5.56/mo. bill credit... Price after additional* $200 in credits (5.56/mo.) *for new consumer lines only."*.
- **Interpretation (Variant Checking):** The text explicitly lists "$660.36 in bill credits" for the "256GB" model. This matches the input product's variant, passing the first validation gate.
- **Interpretation (Line Type Checking):** The overarching terms explicitly state "Eligibility: New customers activating new line". This acts as a strict global gate. If the simulated line type is "New Line," the discounts are valid. If the simulated line type is "Upgrade," the customer is completely blocked from this offer block.
- **Interpretation (Separating Unique vs. Service Line):** For the valid New Line, the offer breaks down into a base device discount ($660.36) and an *additional* service line credit ($200). These must never be summed together; they must be routed to their respective columns.
- **Expected Output (New Line):**
    - UNIQUE DISCOUNT: $660.36
    - SERVICE LINE OFFERS: Price after additional $200 in credits.
    - SERVICE LINE OFFER $ VALUE: $200
- **Expected Output (Upgrade):**
    - UNIQUE DISCOUNT: null
    - SERVICE LINE OFFERS: null
    - SERVICE LINE OFFER $ VALUE: null

3. Max TI (Trade-In) Detection

- **Source Text:** The JSON contains a section titled *"AT&T Trade-In Program"* that states: *"Receive an AT&T Promotional Card when you trade in your previous device."*.
- **Interpretation:** To populate the Max TI field, the text must contain a specific, explicitly stated maximum dollar amount (e.g., "Up to $1000 off") tied to the trade-in action. While the text discusses the trade-in program's existence and condition requirements (e.g.,

"Power on and off", "Have an intact screen"), it does not explicitly state a maximum promotional dollar value in this specific JSON block.

- **Expected Output (Both Line Types):**
    - MAXIMUM TRADE-IN VALUE: null

4. Switcher Offers

- **Source Text:** *"Get up to $800/line to break your contract. Learn how to switch to AT&T and get a reward card to pay off your phone."*.
- **Interpretation:** This offer is explicitly triggered by port-in or carrier-switching behavior. By standard definition, an existing customer doing an "Upgrade" on an already active line cannot switch to a carrier they are already using. This field strictly evaluates for New Lines.
- **Expected Output (New Line):**
    - SWITCHER: $800
    - SWITCHER_OFFER_TEXT: Get up to $800/line when you switch to AT&T and get a reward card to pay off your phone.
- **Expected Output (Upgrade):**
    - SWITCHER: null

5. Finance / Bank Offers

- **Source Text:** *"Earn a $200 statement credit after spending $1,000 on purchases in your first three months... Earn up to $240 in statement credits every year with the AT&T Points Plus Card from Citi... Earn $10 discounts on your AT&T bills"*.
- **Interpretation:** This is a third-party financial institution offer (Citi), distinct from standard carrier installment plans. The calculation logic strictly depends on the Line Type. A "New Line" qualifies for the acquisition welcome bonus ($200), recurring credits ($240), and the explicit bill discount ($10). An "Upgrade" line is strictly excluded from one-time acquisition bonuses and only qualifies for the recurring benefits ($240 + $10 = $250). Spend requirements (like "spending $1,000") are completely ignored in the value calculation.
- **Expected Output (New Line):**
    - FINANCE OFFER DESCRIPTION 1: Earn a $200 statement credit after qualifying purchases welcome bonus plus up to $240 back every year with the AT&T Points Plus® Card from Citi and $10 discounts on your AT&T bills
    - FINANCE OFFER $ VALUE 1: $450
- **Expected Output (Upgrade):**
    - FINANCE OFFER DESCRIPTION 1: Get up to $240 back every year with the AT&T Points Plus® Card from Citi and $10 discounts on your AT&T bills
    - FINANCE OFFER $ VALUE 1: $250

Output Requirements & Data Structure

Your script must process the unstructured JSON input and generate a strictly formatted CSV file. The integrity of the output structure is critical for downstream analytics.

The "Two-Row" Multiplication Rule For every single product object in the source JSON array, your code must generate exactly **two distinct rows** in the final CSV:

- **Row 1:** Evaluated under the assumption that `LINE TYPE` is **"New Line"**.
-
- **Row 2:** Evaluated under the assumption that `LINE TYPE` is **"Upgrade"**.
-

This means if the input JSON contains 10 products, your output CSV must contain exactly 20 rows. You must independently evaluate the promotional text for each line type constraint (e.g., a "New Line" row might extract an $800 Switcher offer, while the "Upgrade" row for the exact same device must output `null` for that field).

Formatting & Schema Specifications

- **File Format:** CSV (Comma Separated Values).

- **Columns:** The output must contain the exact column headers specified in the assignment. Do not add, remove, or rename any columns.

- **Monetary Values:** All fields requiring a dollar value (for example, `MAXIMUM TRADE-IN VALUE`, `SWITCHER`, `UNIQUE DISCOUNT`) must be formatted with a leading dollar sign and exact decimals (for example, `$800`, `$244.36`). Never drop trailing decimals or round numbers

- **Null Handling:** If an offer does not exist for a specific field, or if the Line Type disqualifies the customer from the offer, leave the CSV cell blank (or output a standard `null` equivalent depending on your parsing logic). Do not use placeholders like "N/A", "0", or "-".

- **Non-Duplication:** A single monetary value or promotional snippet from the source text must only be mapped to **one** category column.

Evaluation Criteria

Your submission will be reviewed by our engineering and data teams based on the following dimensions:

1. Extraction Accuracy & Prompt Engineering

- **Line Type Logic:** Did your solution accurately differentiate between "New Line" and "Upgrade" eligibility based on the raw text?
- **Strict Variant Matching:** Did the model correctly anchor discounts to the specific storage variant (e.g., applying a 256GB discount only to the 256GB row)?
- **Hallucination Prevention:** Did the system successfully avoid pulling in external knowledge, guessing missing prices, or rounding exact decimal values?

2. Architecture & Code Quality

- **Pipeline Design:** How effectively does your code bridge the LLM extraction phase with the Python-based normalization/validation phase?
- **Error Handling:** How robust is your JSON parsing? Does your code gracefully handle malformed LLM outputs, timeouts, or unexpected payload structures?
- **Readability:** Is the code modular, well-documented, and easy for another developer to maintain?

How to Submit

- **Code Submission:** Please provide a link to your GitHub repository containing the complete codebase. Ensure the repository is set to **public** so our team can access and review it.
- **Additional Bonus:** Include a PDF document in your repository explaining your solution architecture, the methodology you used to solve the problem, and clear, step-by-step instructions on how to run your code.
- **Deadline:** Two days from your assignment's start date.